Computer System and Architecture

Machine Problem

FLAB3

Sison, Calvin Xanneder S.	Mr. Tejuco
Name of Student	Name of Professor
10/29/2026	10/29/2026
Data Performed	Date Submitted

List the steps that you perform to come up with the program:

1. Cor_tuition.asm was developed through GitHub Codespaces using NASM 32-bit assembly language.
2. The macros and procedures for printing text using Linux system calls are created.
3. I placed the 11 subject codes, their titles, and the unit value in the data segment.
4. Buffer space was allocated for input of enrollment status, student number, name, telephone number, ZIP code, classification, college, program, year level, semester, school year, and address.
5. Procedures that get the registration information, tuition rate per unit, and other fees were included.
6. I added validation of required fields, too big string, and invalid monetary amount. The classification field may be optional.
7. Procedure compute_tuition is included which adds up all the units, multiplies it to the tuition rate and other fees.
8. I printed the entered registration information, subjects, total units, tuition fee, other fees, and total assessment.
9. The source was compiled using NASM and linked using ld before executing the program.
10. Values were entered and the output verified. A total assessment of PHP 22,500 was obtained from 21 units, PHP 1,000 tuition rate, and PHP 1,500 other fees.

Paste the source code and explain the functionalities and take a screenshot of the final output.

Source code:

```
; FLAB3 - Certificate of Registration and Tuition Calculator
; NASM x86 32-bit Linux. Amounts are stored as integer centavos.

%macro print_text 2
    mov ecx, %1
    mov edx, %2
```

```
    call print_proc
%endmacro

section .data
    banner db 10, 'COR AND TUITION FEE CALCULATOR', 10, 0
    banner_len equ $-banner-1
    input_note db 'Enter the registration details below.', 10
              db 'Classification may be left blank by pressing Enter.', 10, 10, 0
    ask_rate db 'Tuition rate per unit (PHP): ', 0
    ask_other db 'Other fees (PHP, enter 0 if none): ', 0
    bad_text db 'Invalid entry. Please try again: ', 0
    bad_money db 'Enter an amount from 0 to 99999.99, with up to 2 decimal places.', 10, 0
    bad_rate db 'The tuition rate must be greater than zero.', 10, 0
    heading db 10,
'============================================================', 10
            db '                CERTIFICATE OF REGISTRATION', 10
            db '============================================================',
10, 0
    status_label db 'Enrollment status : ', 0
    id_label db 'Student number    : ', 0
    name_label db 'Student name      : ', 0
    phone_label db 'Phone number      : ', 0
    zip_label db 'ZIP code         : ', 0
    class_label db 'Classification    : ', 0
    college_label db 'College          : ', 0
    program_label db 'Program          : ', 0
    year_label db 'Year level       : ', 0
    term_label db 'Term             : ', 0
    sy_label db 'School year      : ', 0
    address_label db 'Address          : ', 0
    ; Each record: label, destination, capacity, allow-empty flag.
    field_table dd status_label, enrollment_status, 32, 0
                dd id_label, student_id, 32, 0
                dd name_label, student_name, 128, 0
                dd phone_label, phone_number, 32, 0
                dd zip_label, zip_code, 16, 0
                dd class_label, classification, 64, 1
                dd college_label, college_name, 96, 0
                dd program_label, program_name, 64, 0
                dd year_label, year_level, 32, 0
                dd term_label, school_term, 64, 0
                dd sy_label, school_year, 32, 0
                dd address_label, student_address, 192, 0
    courses db 'COURSE CODE  COURSE TITLE                              UNITS', 10
            db '------------------------------------------------------------', 10
            db 'CCS0021      Information Management (LEC)                2', 10
            db 'CCS0021L     Information Management (LAB)                1', 10
            db 'CS0001       Discrete Structures 1                       3', 10
            db 'CS0003       Computer Systems and Architecture           2', 10
            db 'CS0003L      Computer Systems and Architecture (LAB)     1', 10
            db 'CS0070       Object Oriented Programming                 2', 10
```

```
        db 'CS0070L     Object Oriented Programming (LAB)          1', 10
        db 'GED0021     Specialized English Program 2              3', 10
        db 'GED0075     Linear Algebra                             3', 10
        db 'GED0081     College Physics 1 Lecture                  2', 10
        db 'GED0081L    College Physics 1 Laboratory               1', 10
        db '------------------------------------------------------------', 10, 0
    unit_values db 2, 1, 3, 2, 1, 2, 1, 3, 3, 2, 1
    units_label db 'Total enrolled units : ', 0
    rate_label db 'Rate per unit       : PHP ', 0
    tuition_label db 'Tuition fee        : PHP ', 0
    other_label db 'Other fees         : PHP ', 0
    total_label db 'TOTAL ASSESSMENT    : PHP ', 0
    formula db 10, 'Tuition fee = total units x rate per unit', 10
        db 'Total assessment = tuition fee + other fees', 10, 0
    rule db '============================================================',
10, 0
    newline db 10, 0
    dot db '.'

section .bss
    enrollment_status resb 32
    student_name resb 128
    student_id resb 32
    phone_number resb 32
    zip_code resb 16
    classification resb 64
    college_name resb 96
    program_name resb 64
    year_level resb 32
    school_term resb 64
    school_year resb 32
    student_address resb 192
    input_buffer resb 64
    input_char resb 1
    number_buffer resb 16
    cents_buffer resb 2
    total_units resd 1
    rate_cents resd 1
    other_cents resd 1
    tuition_cents resd 1
    assessment_cents resd 1

section .text
    global _start

_start:
    print_text banner, banner_len
    call get_student_info

    mov ecx, ask_rate
    mov ebx, 1
    call get_money
```

```
    mov [rate_cents], eax
    mov ecx, ask_other
    xor ebx, ebx
    call get_money
    mov [other_cents], eax

    call compute_tuition
    call display_cor
    jmp exit_program

; Collect all twelve header fields. Only classification may be empty.
get_student_info:
    pushad
    mov ecx, input_note
    call print_z
    mov edi, field_table
    mov ebp, 12
.next:
    mov ecx, [edi]
    call print_z
    mov ecx, [edi+4]
    mov edx, [edi+8]
    call read_line
    jc .invalid
    test eax, eax
    jnz .accepted
    cmp dword [edi+12], 1
    jne .invalid
.accepted:
    add edi, 16
    dec ebp
    jnz .next
    popad
    ret
.invalid:
    mov ecx, bad_text
    call print_z
    jmp .next

; Display exactly the values entered by the user.
display_student_info:
    pushad
    mov edi, field_table
    mov ebp, 12
.next:
    mov ecx, [edi]
    call print_z
    mov ecx, [edi+4]
    call print_z
    mov ecx, newline
    call print_z
    add edi, 16
```

```
    dec ebp
    jnz .next
    mov ecx, newline
    call print_z
    popad
    ret

; ECX = text address, EDX = length. Used by the macro and print_z.
print_proc:
    pushad
    mov eax, 4
    mov ebx, 1
    int 0x80
    popad
    ret

; Print a zero-terminated string; preserve all general-purpose registers.
print_z:
    pushad
    xor edx, edx
.length:
    cmp byte [ecx+edx], 0
    je .write
    inc edx
    jmp .length
.write:
    call print_proc
    popad
    ret

; Sum the course units, multiply by the rate, then add other fees.
compute_tuition:
    xor eax, eax
    mov esi, unit_values
    mov ecx, 11
.sum:
    movzx edx, byte [esi]
    add eax, edx
    inc esi
    loop .sum
    mov [total_units], eax
    mul dword [rate_cents]
    mov [tuition_cents], eax
    add eax, [other_cents]
    mov [assessment_cents], eax
    ret

display_cor:
    mov ecx, heading
    call print_z
    call display_student_info
    mov ecx, courses
```

```
    call print_z
    mov ecx, units_label
    call print_z
    mov eax, [total_units]
    call print_number
    mov ecx, newline
    call print_z
    mov ecx, rate_label
    call print_z
    mov eax, [rate_cents]
    call print_money
    mov ecx, tuition_label
    call print_z
    mov eax, [tuition_cents]
    call print_money
    mov ecx, other_label
    call print_z
    mov eax, [other_cents]
    call print_money
    mov ecx, total_label
    call print_z
    mov eax, [assessment_cents]
    call print_money
    mov ecx, formula
    call print_z
    mov ecx, rule
    call print_z
    ret

; Print EAX as an unsigned decimal integer.
print_number:
    pushad
    mov edi, number_buffer+16
    mov ebx, 10
    xor ecx, ecx
.digit:
    xor edx, edx
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    inc ecx
    test eax, eax
    jnz .digit
    mov edx, ecx
    mov ecx, edi
    call print_proc
    popad
    ret

; Print EAX centavos as pesos with exactly two decimal places.
```

```
print_money:
    pushad
    mov ebx, 100
    xor edx, edx
    div ebx
    push edx
    call print_number
    print_text dot, 1
    pop eax
    xor edx, edx
    mov ebx, 10
    div ebx
    add al, '0'
    add dl, '0'
    mov [cents_buffer], al
    mov [cents_buffer+1], dl
    print_text cents_buffer, 2
    mov ecx, newline
    call print_z
    popad
    ret

; ECX = prompt, EBX = minimum centavos (1 for rate, 0 for other fees).
get_money:
    push esi
    push edi
    mov edi, ecx
    mov esi, ebx
.retry:
    mov ecx, edi
    call print_z
    mov ecx, input_buffer
    mov edx, 64
    call read_line
    jc .bad
    call parse_money
    jc .bad
    cmp eax, esi
    jb .zero_rate
    pop edi
    pop esi
    ret
.zero_rate:
    mov ecx, bad_rate
    call print_z
    jmp .retry
.bad:
    mov ecx, bad_money
    call print_z
    jmp .retry
```

```
; Accept digits with an optional dot and 1-2 fractional digits.
; Return EAX centavos, CF=1 if malformed or above PHP 99999.99.
parse_money:
    push ebx
    push esi
    push edi
    mov esi, input_buffer
    xor eax, eax
    mov edi, -1
    cmp byte [esi], 0
    je .bad
.next:
    movzx ebx, byte [esi]
    test ebx, ebx
    jz .finish
    cmp bl, '.'
    je .decimal
    sub ebx, '0'
    cmp ebx, 9
    ja .bad
    imul eax, eax, 10
    add eax, ebx
    cmp eax, 9999999
    ja .bad
    cmp edi, -1
    je .advance
    inc edi
    cmp edi, 2
    ja .bad
.advance:
    inc esi
    jmp .next
.decimal:
    cmp esi, input_buffer
    je .bad
    cmp edi, -1
    jne .bad
    xor edi, edi
    jmp .advance
.finish:
    cmp edi, 0
    je .bad
    cmp edi, -1
    jne .fraction
    imul eax, eax, 100
    jmp .range
.fraction:
    cmp edi, 1
    jne .range
    imul eax, eax, 10
.range:
```

```
    cmp eax, 9999999
    ja .bad
    clc
    jmp .return
.bad:
    stc
.return:
    pop edi
    pop esi
    pop ebx
    ret

; Read one complete line, strip CRLF, drain oversized lines.
; ECX=destination, EDX=capacity. Return EAX=length and CF=invalid.
read_line:
    push ebx
    push ecx
    push edx
    push esi
    push edi
    push ebp
    mov esi, ecx
    mov ebp, edx
    dec ebp
    xor edi, edi
.read:
    mov eax, 3
    xor ebx, ebx
    mov ecx, input_char
    mov edx, 1
    int 0x80
    test eax, eax
    js exit_program
    jz .eof
    mov al, [input_char]
    cmp al, 10
    je .finish
    test al, al
    jz .drain
    cmp edi, ebp
    jae .drain
    mov [esi+edi], al
    inc edi
    jmp .read
.eof:
    test edi, edi
    jz exit_program
.finish:
    test edi, edi
    jz .terminate
    cmp byte [esi+edi-1], 13
```

```
    jne .terminate
    dec edi
.terminate:
    mov byte [esi+edi], 0
    mov eax, edi
    clc
    jmp .return
.drain:
    mov eax, 3
    xor ebx, ebx
    mov ecx, input_char
    mov edx, 1
    int 0x80
    test eax, eax
    jle .bad
    cmp byte [input_char], 10
    jne .drain
.bad:
    mov byte [esi+edi], 0
    stc
.return:
    pop ebp
    pop edi
    pop esi
    pop edx
    pop ecx
    pop ebx
    ret

exit_program:
    mov eax, 1
    xor ebx, ebx
    int 0x80
```

Explanation of Functionalities:

- The program collects registration information and displays a Certificate of Registration with a tuition assessment.
- The print_text macro expands into instructions that prepare the text address and length before calling print_proc. The print_proc procedure displays the text using int 0x80 and returns using RET.
- The get_student_info procedure collects all 12 registration fields. The read_line procedure reads complete lines and rejects oversized input. Classification is optional.

- The get_money and parse_money procedures validate the tuition rate and other fees. Monetary values are stored as integer centavos to calculate amounts accurately without floating-point arithmetic.
- The compute_tuition procedure adds the units of all 11 subjects, producing 21 units. It multiplies the total units by the rate per unit, then adds other fees to obtain the total assessment.
- The display_student_info and display_cor procedures print the entered information and assessment. The print_number and print_money procedures format numeric values, with monetary amounts displayed to two decimal places.
- The exit_program procedure terminates the application using the Linux exit system call.

Output Screenshots:

```
COR AND TUITION FEE CALCULATOR
Enter the registration details below.
Classification may be left blank by pressing Enter.

Enrollment status : ENROLLED
Student number    : 202511575
Student name      : SISON, CALVIN XANNEDER SOMERA
Phone number      : +639176885502
ZIP code          : 829
Classification    :
College           : COMPUTER STUDIES
Program           : BSCSSSE
Year level        : 2
Term              : 1st
School year       : 26-27
Address           : 123 Hollowood Avenue
Tuition rate per unit (PHP): 1000
Other fees (PHP, enter 0 if none): 1500
```

```
================================================================
                  CERTIFICATE OF REGISTRATION
================================================================
Enrollment status : ENROLLED
Student number    : 202511575
Student name      : SISON, CALVIN XANNEDER SOMERA
Phone number      : +639176885502
ZIP code          : 829
Classification    :
College           : COMPUTER STUDIES
Program           : BSCSSSE
Year level        : 2
Term              : 1st
School year       : 26-27
Address           : 123 Hollowood Avenue
```

```
COURSE CODE  COURSE TITLE                                  UNITS
----------------------------------------------------------------
CCS0021      Information Management (LEC)                    2
CCS0021L     Information Management (LAB)                    1
CS0001       Discrete Structures 1                           3
CS0003       Computer Systems and Architecture               2
CS0003L      Computer Systems and Architecture (LAB)         1
CS0070       Object Oriented Programming                     2
CS0070L      Object Oriented Programming (LAB)               1
GED0021      Specialized English Program 2                   3
GED0075      Linear Algebra                                  3
GED0081      College Physics 1 Lecture                       2
GED0081L     College Physics 1 Laboratory                    1
----------------------------------------------------------------
Total enrolled units : 21
Rate per unit        : PHP 1000.00
Tuition fee          : PHP 21000.00
Other fees           : PHP 1500.00
TOTAL ASSESSMENT     : PHP 22500.00

Tuition fee = total units x rate per unit
Total assessment = tuition fee + other fees
================================================================
```